NIDS Project — Complete Work Summary

Network Intrusion Detection System using Machine Learning and Artificial Neural Network

Dataset: NF-UNSW-NB15 | Environment: Python + Anaconda | Python 3.13.5 | TensorFlow 2.21.2

1. Executive Summary

We built a binary Network Intrusion Detection System (NIDS) that classifies network flows as Benign (0) or Attack (1). The project started with a Logistic Regression baseline, then addressed class imbalance using class-weighted Logistic Regression, and finally developed an Artificial Neural Network (ANN). The ANN was trained on standardized flow-level features and evaluated on a completely held-out test set of 40,000 samples.

The most important result at the default 0.50 decision threshold was 96.02% test accuracy, 97.32% attack recall, 62.93% attack precision, 76.44% attack F1, 4.07% false-positive rate, and 2.68% false-negative rate. We then performed threshold analysis from 0.30 to 0.70. At threshold 0.70, the model reached 70.88% precision, 91.93% recall, 80.05% F1 and 2.68% FPR. The 0.70 operating point was identified as a strong balance for a NIDS, although its final accuracy and confusion matrix should be explicitly recalculated before being reported as the final operating point.

2. Project Goal

The goal is to build a practical NIDS that can learn patterns from network-flow statistics and flag suspicious traffic. The key cybersecurity requirement is not merely high overall accuracy: the system should detect a large proportion of actual attacks while keeping false alarms manageable.

- Primary task: binary classification — 0 = Benign, 1 = Attack.
- Main success metric for security: Attack Recall / True Positive Rate.
- Important supporting metrics: Precision, F1, False Positive Rate (FPR), and False Negative Rate (FNR).
- Secondary analysis: determine how well the ANN detects individual attack families.

3. Dataset and Initial File Handling

The dataset used was NF-UNSW-NB15.xlsx. The Excel workbook was large enough that opening it manually inside the Jupyter interface produced an error/was inconvenient. The file was kept in the same Jupyter working folder as the notebook.

Item	Observed value
File	NF-UNSW-NB15.xlsx
Workbook size	70.36 MB
Sheet	NF-UNSW-NB15

Rows in full sheet	1,048,576
Columns in full sheet	14
Rows actually loaded for modeling	200,000
Columns used for modeling	10 features + Label target

Because the full Excel sheet is very large, the modeling workflow used a 200,000-row portion. This kept the experiments practical on the available laptop while still providing a substantial dataset.

4. Dataset Columns

Column	Meaning	Dtype
IPV4_SRC_ADDR	Source IPv4 address	Object
L4_SRC_PORT	Source transport-layer port	int64
IPV4_DST_ADDR	Destination IPv4 address	Object
L4_DST_PORT	Destination transport-layer port	int64
PROTOCOL	Network/transport protocol identifier	int64
L7_PROTO	Application-layer protocol identifier	float64
IN_BYTES	Bytes received/inbound	int64
OUT_BYTES	Bytes sent/outbound	int64
IN_PKTS	Inbound packet count	int64
OUT_PKTS	Outbound packet count	int64
TCP_FLAGS	TCP flag information	int64
FLOW_DURATION_MILLISECONDS	Duration of the network flow	int64
Label	Binary target: 0 benign, 1 attack	int64
Attack	Attack family/category label	Object

The 200,000-row DataFrame inspection showed:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 14 columns): 200000 non-null for every column
dtypes: float64(1), int64(10), object(3)
memory usage: about 21.4+ MB
```

5. Class Distribution and Imbalance

The target variable was highly imbalanced. Most flows were benign, while attacks represented a much smaller class.

Class	Count	Approx. share
Benign (0)	186,734	93.37%
Attack (1)	13,266	6.63%
Total	200,000	100%

Attack-family distribution in the 200,000-row modeling dataset:

Attack type	Count
Exploits	4,765
Fuzzers	3,248
Reconnaissance	2,332
Generic	1,009
DoS	942
Backdoor	349
Analysis	334
Shellcode	258

Worms	29
Benign	186,734

This imbalance is important: a model can obtain high overall accuracy simply by favoring the benign class. Therefore, accuracy alone is not sufficient for judging a NIDS.

6. Feature Selection, X/y, and Train-Test Split

For the binary ML task, the IP address columns and the textual Attack column were not included as model features. The selected numerical flow features were:

- L4_SRC_PORT
- L4_DST_PORT
- PROTOCOL
- L7_PROTO
- IN_BYTES
- OUT_BYTES
- IN_PKTS
- OUT_PKTS
- TCP_FLAGS
- FLOW_DURATION_MILLISECONDS

The resulting shapes were:

```
X shape: (200000, 10)
y shape: (200000,)
```

A 75/25 train-test split was used:

```
X_train: (160000, 10)
X_test : (40000, 10)
y_train: (160000,)
y_test : (40000,)
```

The 40,000-row test set was kept separate for final evaluation. It was not used to update model weights.

7. Feature Scaling

StandardScaler was applied to the numerical feature matrix. Scaling is especially useful because network features have very different magnitudes: ports are in the thousands, byte counts can be much larger, while flags/protocol values are relatively small.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
X_train_scaled shape: (160000, 10)
X_test_scaled shape : (40000, 10)
```

The scaler was fitted only on X_train and then applied to X_test. This avoids leaking information from the test set.

8. Machine Learning Baseline — Logistic Regression

The first model was a standard Logistic Regression classifier. This served as a simple ML baseline against which the ANN could be compared.

The baseline achieved 94.03% accuracy, but its attack recall was only 15%. This exposed the class-imbalance problem: the model was very good at recognizing benign traffic but missed most attacks.

Metric	Normal Logistic Regression
Accuracy	94.03%
Attack precision	75%
Attack recall	15%
Attack F1	25%

This was a critical lesson: 94% accuracy sounds high, but for a NIDS it is not acceptable if the system detects only 15% of actual attacks.

9. Class-Weighted / Balanced Logistic Regression

To address the imbalance, class weighting was used so that errors on the minority Attack class carried much more weight.

The balanced Logistic Regression produced approximately 70% overall accuracy but increased attack recall to 89%. Its attack precision was about 17% and attack F1 about 28%.

Metric	Balanced Logistic Regression
Accuracy	70.00%
Attack precision	17%
Attack recall	89%
Attack F1	28%

Its confusion matrix was:

```
[[25598 11749]
 [  280  2373]]
```

This model showed the trade-off clearly: it detected many more attacks, but it classified a large amount of benign traffic as attacks. The ANN was introduced to seek a better balance.

10. Software Environment

The implementation was developed in Python 3.13.5 using TensorFlow 2.21.2 for the ANN and standard scikit-learn components for preprocessing and baseline ML models.

11. ANN Architecture

The final ANN architecture used 10 input features and three Dense layers:

Layer	Neurons	Activation	Purpose
Input / Dense	64	ReLU	Learn nonlinear patterns from the 10 input features
Dense	32	ReLU	Learn higher-level combinations of features
Output	1	Sigmoid	Output attack probability for binary classification

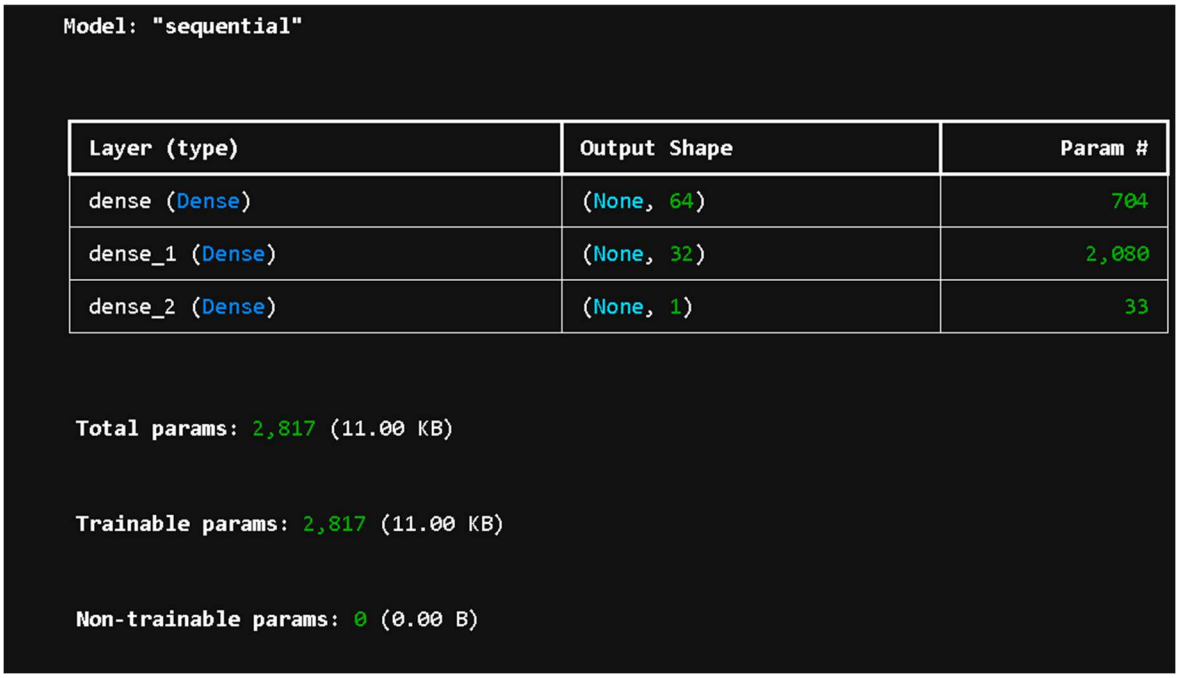
Architecture:

```
10 input features
      ↓
Dense(64) + ReLU
      ↓
Dense(32) + ReLU
      ↓
Dense(1) + Sigmoid
      ↓
Attack probability
```

Model summary showed:

The model has 704 parameters in Dense(64), 2,080 in Dense(32), and 33 in the sigmoid output layer, for a total of 2,817 trainable parameters.

Figure: ANN model summary — layer output shapes, parameter counts, and trainable parameters.



Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	704
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 1)	33

Total params: 2,817 (11.00 KB)

Trainable params: 2,817 (11.00 KB)

Non-trainable params: 0 (0.00 B)

```
Dense        output: (None, 64)
Dense_1      output: (None, 32)
Dense_2      output: (None, 1)

Total parameters: 2,817
Trainable parameters: 2,817
Non-trainable parameters: 0
```

The model was compiled with the Adam optimizer, binary cross-entropy loss, and accuracy as a training metric.

```
ann_model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

12. ANN Class Weighting

Because the Attack class was much smaller than the Benign class, balanced class weights were calculated from y_train.

```
from sklearn.utils.class_weight import compute_class_weight
import numpy as np

class_weights = compute_class_weight(
    class_weight="balanced",
    classes=np.unique(y_train),
    y=y_train
)

class_weight_dict = dict(
    zip(np.unique(y_train), class_weights)
)
Class weights:
0 (Benign)  ≈ 0.5355
1 (Attack)  ≈ 7.5379
```

The Attack class therefore received a much larger training weight. This encourages the ANN to treat missed attacks as much more important than mistakes on the majority benign class.

13. ANN Training

The ANN was trained for 20 epochs with a batch size of 256, a 20% validation split, and the calculated class weights.

```
history = ann_model.fit(
    X_train_scaled,
    y_train,
    epochs=20,
    batch_size=256,
    validation_split=0.2,
    class_weight=class_weight_dict,
    verbose=1
)
```

The 160,000 training samples were internally divided into approximately 128,000 actual training samples and 32,000 validation samples.

Final epoch:

```
Epoch 20/20
accuracy:     0.9591
loss:         0.1156
val_accuracy: 0.9611
val_loss:     0.1004
```

Training and validation accuracy were close at the end of training. This is a good sign and does not show an obvious large persistent overfitting gap.

14. Training vs Validation Accuracy Plot

A plot of training and validation accuracy was created from the Keras history object. The curves converge near 96% toward the end of training. The validation curve fluctuates during some epochs but ends close to the training curve.

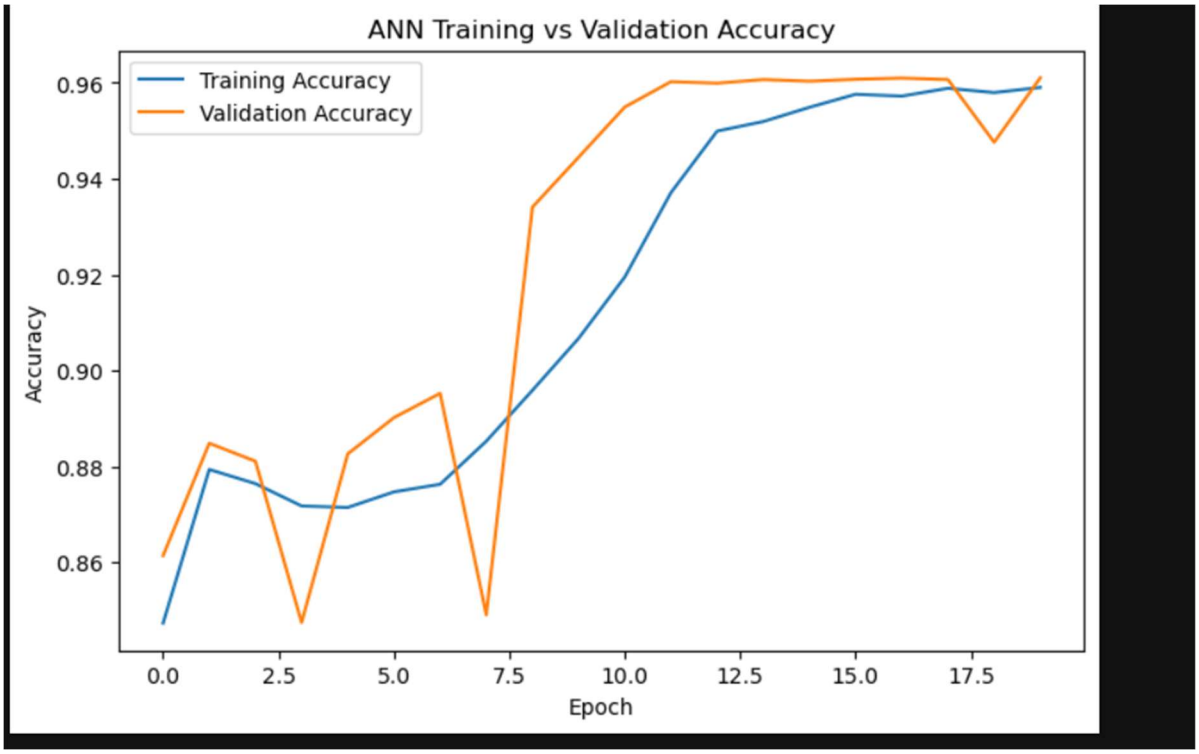
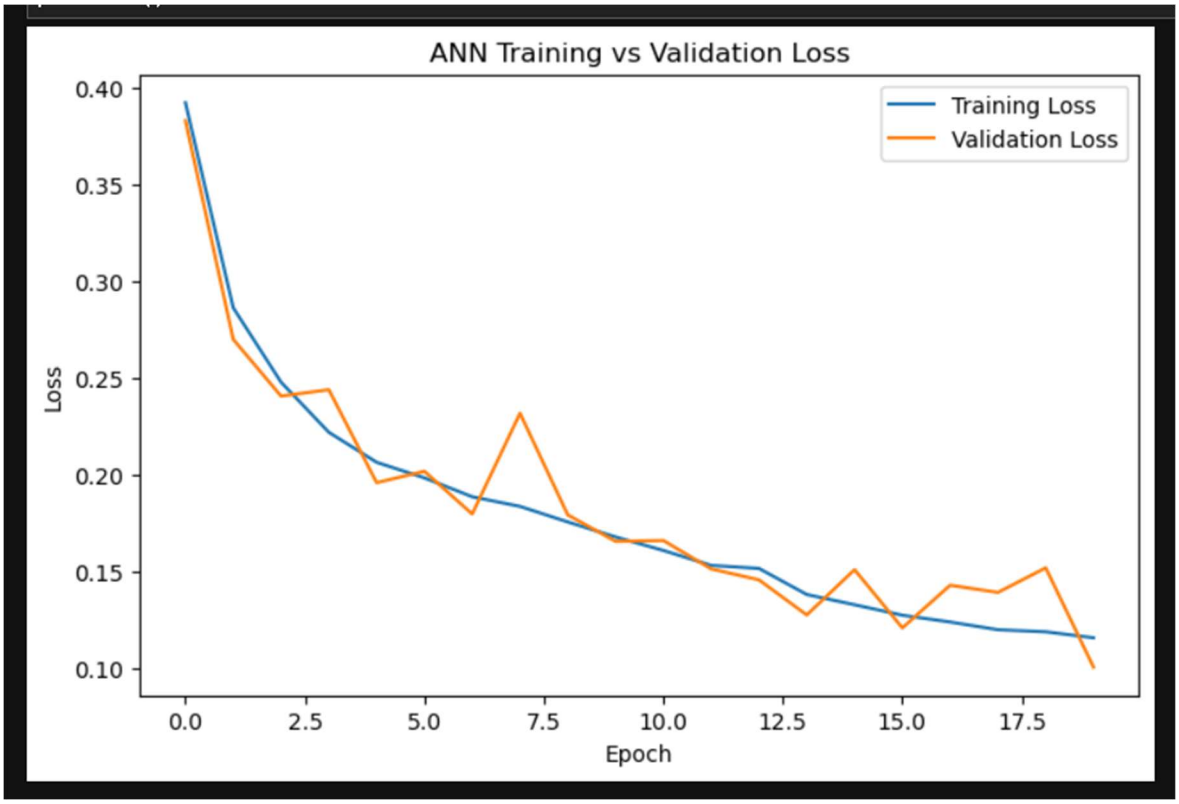


Figure: ANN Training vs Validation Accuracy

Training and validation loss generally decrease over the 20 epochs, with some validation fluctuations. The curves finish close together, supporting the observation that there is no obvious severe overfitting.

Figure: ANN Training vs Validation Loss over 20 epochs.



15. ANN Test Evaluation at Threshold 0.50

The ANN produced probabilities with sigmoid output. A default threshold of 0.50 was first used:

```
y_prob_ann = ann_model.predict(X_test_scaled)

y_pred_ann = (y_prob_ann >= 0.5).astype(int).ravel()
```

The rule is: probability < 0.50 → Benign; probability ≥ 0.50 → Attack.

The test classification report was:

Class	Precision	Recall	F1	Support
Benign (0)	1.00	0.96	0.98	37,347
Attack (1)	0.63	0.97	0.76	2,653
Overall accuracy	—	—	0.96	40,000
Macro average	0.81	0.97	0.87	40,000
Weighted average	0.97	0.96	0.96	40,000

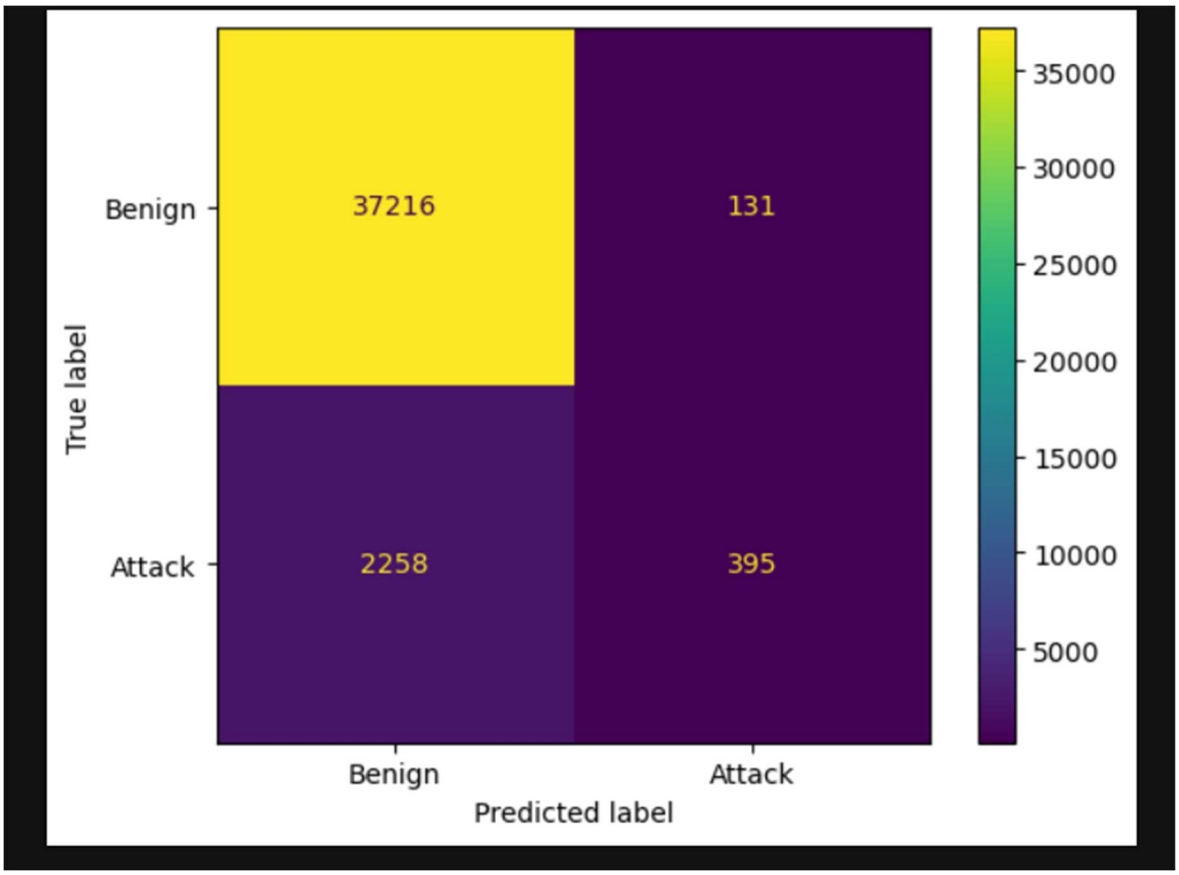
Confusion matrix at threshold 0.50:

```
[[35826 1521]
 [   71 2582]]
```

Interpretation: 35,826 benign flows were correctly classified, 1,521 benign flows were falsely flagged as attacks, 71 attacks were missed, and 2,582 attacks were correctly detected.

This screenshot shows TN=37,216, FP=131, FN=2,258, and TP=395. Because these values differ from the separately recorded 0.50-threshold matrix [[35826, 1521], [71, 2582]], this image should be treated as the confusion matrix for the notebook operating point that generated the screenshot, rather than being substituted into the 0.50 results table.

Figure: ANN confusion matrix shown in the supplied notebook output.



16. NIDS-Specific Metrics at Threshold 0.50

Metric	Value	Interpretation
Accuracy	96.02%	Overall fraction classified correctly
Precision	62.93%	About 63% of attack alerts were actual attacks
Recall / TPR	97.32%	About 97% of actual attacks were detected
F1 Score	76.44%	Balance of attack precision and recall
FPR	4.07%	About 4% of benign flows were falsely flagged
FNR	2.68%	About 2.7% of attacks were missed

For a NIDS, the 97.32% attack recall is especially important because it means the system detected 2,582 of the 2,653 actual attacks in the test set and missed only 71.

17. Attack-Family Analysis

The binary target only distinguishes Benign vs Attack. To understand the model more deeply, the Attack column was attached to the test predictions and the recall of each attack family was calculated.

Attack type	Detected correctly	Test total	Recall
Analysis	62	62	100.00%
Shellcode	49	49	100.00%
Worms	7	7	100.00%*
Reconnaissance	464	464	100.00%
Fuzzers	703	704	99.86%
Backdoor	58	59	98.31%
Generic	179	188	95.21%
Exploits	876	924	94.81%
DoS	184	196	93.88%

* The Worms result is based on only 7 test samples. It is correct to say the model achieved 100% recall on those seven samples, but it would not be scientifically appropriate to claim universal 100% Worm detection from such a small sample.

The strongest large-sample categories were Reconnaissance (100%), Fuzzers (99.86%), Backdoor (98.31%), Exploits (94.81%), and DoS (93.88%).

18. Decision Threshold Analysis

The sigmoid output is a probability rather than a fixed class. Instead of automatically using 0.50, several thresholds were tested to understand the precision/recall/false-positive trade-off.

Threshold	Precision	Recall	F1	FPR
0.30	62.17%	98.98%	76.37%	4.28%
0.40	62.30%	98.12%	76.21%	4.22%
0.50	62.93%	97.32%	76.44%	4.07%
0.60	63.83%	96.53%	76.85%	3.89%
0.70	70.88%	91.93%	80.05%	2.68%

Increasing the threshold makes the model more conservative about declaring a flow to be an attack. Consequently, precision and F1 improve while recall decreases. False-positive rate also decreases.

A 0.70 threshold was identified as a strong candidate for a balanced NIDS operating point because it had the highest F1 among the tested thresholds (80.05%) and reduced FPR to 2.68%, while still retaining 91.93% attack recall.

Important: the 0.70 threshold metrics above were calculated for precision/recall/F1/FPR. The final 0.70 accuracy and confusion matrix were not explicitly recalculated in the conversation, so those should be generated before the 0.70 operating point is presented as the final model.

```
final_threshold = 0.70
y_pred_final = (y_prob_ann >= final_threshold).astype(int).ravel()

# Then calculate:
# accuracy, precision, recall, F1, FPR, FNR
# and the final confusion matrix.
```

If the priority is maximum attack detection, 0.30 gives 98.98% recall. If the priority is a more balanced alerting system, 0.70 is the more attractive tested point.

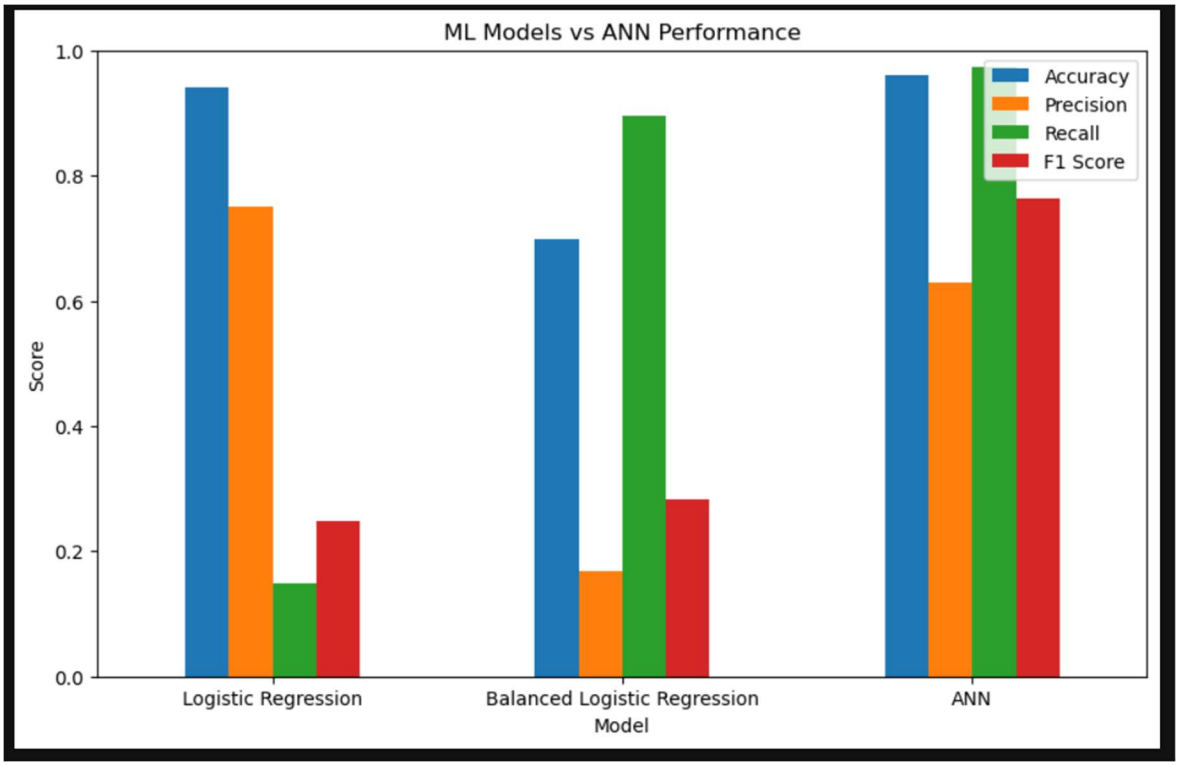
19. ML vs ANN — What We Learned

Model	Accuracy	Attack Recall	Attack F1	Main behavior
Normal Logistic Regression	94.03%	15%	25%	High overall accuracy but misses most attacks
Balanced Logistic Regression	70.00%	89%	28%	Much better attack detection but many false positives
ANN @ 0.50	96.02%	97.32%	76.44%	High attack detection with substantially better balance
ANN @ 0.70	Not recalculated	91.93%	80.05%	Best tested F1 and lower FPR; final accuracy should be calculated

The key conclusion is that the ANN did not simply increase accuracy. It substantially improved the security-relevant ability to detect attacks while avoiding the extreme false-positive trade-off observed with the balanced Logistic Regression.

The plot visually shows the central result: normal Logistic Regression has high accuracy but poor attack recall; balanced Logistic Regression increases attack recall at the cost of many false positives; and the ANN provides the strongest combined performance among the plotted models.

Figure: ML Models vs ANN Performance — Accuracy, Precision, Recall, and F1.



20. Confusion Matrix Concepts Used

Term	Meaning in this NIDS
TN	Benign traffic correctly classified as Benign
FP	Benign traffic incorrectly flagged as Attack
FN	Attack traffic incorrectly classified as Benign — missed attack
TP	Attack traffic correctly classified as Attack

At threshold 0.50:

```
TN = 35,826
FP = 1,521
FN = 71
TP = 2,582
```

The formulas used were: Recall = TP/(TP+FN), Precision = TP/(TP+FP), FPR = FP/(FP+TN), FNR = FN/(FN+TP), and F1 = 2 × Precision × Recall / (Precision + Recall).

21. Complete NIDS Pipeline

```
NF-UNSW-NB15 Dataset
        ↓
Load and inspect Excel data
        ↓
Use 200,000 rows for practical modeling
        ↓
Select 10 numerical flow features
        ↓
```

```
Binary target: Label
0 = Benign, 1 = Attack
        ↓
Train/Test Split
160,000 / 40,000
        ↓
StandardScaler
        ↓
Baseline Logistic Regression
        ↓
Balanced Logistic Regression
        ↓
ANN with class weighting
        ↓
Dense 64 + ReLU
        ↓
Dense 32 + ReLU
        ↓
Dense 1 + Sigmoid
        ↓
Attack probability
        ↓
Threshold analysis
        ↓
Benign / Attack decision
        ↓
NIDS evaluation
Accuracy, Precision, Recall, F1, FPR, FNR
        ↓
Attack-family analysis
Exploits, Fuzzers, DoS, etc.
```

Conceptually, the trained ANN takes a network flow's numerical characteristics and converts them into an attack probability. A threshold then converts that probability into the final binary alert decision.

22. Important Limitations and Scientific Notes

- The full Excel sheet contains 1,048,576 rows, while 200,000 rows were used for the modeling workflow. Results therefore describe this modeling subset, not automatically the entire workbook.
- The dataset is strongly imbalanced toward Benign traffic.
- Some attack families have very few test samples, especially Worms (7), so category-level 100% recall should not be generalized.
- Accuracy alone is misleading for NIDS evaluation; attack recall, F1, FPR, and FNR are more informative.
- Threshold 0.70 looks attractive from the tested precision/recall/F1/FPR trade-off, but its final accuracy/confusion matrix must be explicitly calculated before calling it the final operating point.
- The ANN is a binary classifier. It detects whether traffic is an attack, not which attack family it is directly. The Attack column was used afterward for analysis of binary predictions.

23. Final Takeaway

The project successfully moved from a simple ML baseline to a class-aware ANN-based NIDS. The most important achievement was not simply obtaining approximately 96% accuracy. It was demonstrating that the ANN could detect approximately 97% of actual attacks at the default threshold while maintaining a much better overall balance than the balanced Logistic Regression. Threshold analysis further showed that the operating point can be tuned depending on whether the deployment prioritizes maximum attack detection or fewer false alarms.

The next practical phase is to finalize the 0.70 threshold evaluation, create the final comparison/visualizations, and turn the completed pipeline into the project report, flowchart, and professor presentation.